

Práctica N° 2 - Node.js + Express + MySQL

Materia: INF-133 Programación Web 3

Carrera: Informática – UMSA

Gestión I/2026

1. Enunciado de la Práctica

- Crea un endpoint POST /categorias que permita registrar una nueva categoría enviando nombre y descripción en el body de la petición.
- Crea un endpoint GET /categorias que devuelva todas las categorías registradas en la base de datos.
- Crea un endpoint GET /categorias/:id que devuelva la categoría con el ID especificado y además, incluya todos los productos que pertenecen a esa categoría.
- Crea un endpoint PATCH /categorias/:id que permita actualizar todos los datos de la categoría con el ID especificado.
- Crea un endpoint DELETE /categorias/:id que elimine la categoría indicada y, al mismo tiempo, elimine automáticamente todos los productos que pertenecen a esa categoría.

Paso 1: Crear Base de Datos

```
CREATE DATABASE practica2;
```

```
USE practica2;
```

```
CREATE TABLE categorias (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  nombre VARCHAR(100) NOT NULL,
```

```
  descripcion VARCHAR(255),
```

```
  createdAt DATETIME NOT NULL DEFAULT current_timestamp(),
```

```
  updatedAt DATETIME NOT NULL DEFAULT current_timestamp()
```

```
);
```

```
INSERT INTO categorias (nombre, descripcion)
```

```
VALUES
```

```
('Electrónica', 'Dispositivos electrónicos y gadgets'),
```

```
('Oficina', 'Material y accesorios de oficina');
```

```
CREATE TABLE productos(
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  nombre VARCHAR(100),
```

```
  precio DECIMAL(10,2),
```

```
  categoria_id INT,
```

```
  FOREIGN KEY (categoria_id)
```

```
  REFERENCES categorias(id)
```

```
  ON DELETE CASCADE
```

Categoria.js

```
const express = require('express');
const router = express.Router();
const db = require('../db');
router.get('/', (req, res) => {
  const sql = 'SELECT * FROM categorias';
  db.query(sql, (err, result) => {
    if(err){
      return res.status(500).json(err);
    }
    res.json(result);
  });
});
// POST NUEVA CATEGORIA
router.post('/', (req, res) => {
  const { nombre, descripcion } = req.body;
  const sql = `
    INSERT INTO categorias(nombre, descripcion)
    VALUES (?, ?)
  `;
  db.query(sql, [nombre, descripcion], (err, result) => {
    if(err){
      return res.status(500).json(err);
    }
    res.json({
      mensaje: 'Categoría registrada',
      result
    });
  });
});
// GET POR ID
```

```

router.get('/:id', (req, res) => {
  const id = req.params.id;
  const sql = `
    SELECT * FROM categorias
    WHERE id = ?
  `;
  db.query(sql, [id], (err, result) => {
    if(err){
      return res.status(500).json(err);
    }
    res.json(result);
  });
});

// PATCH ACTUALIZAR
router.patch('/:id', (req, res) => {
  const id = req.params.id;
  const { nombre, descripcion } = req.body;
  const sql = `
    UPDATE categorias
    SET nombre = ?, descripcion = ?
    WHERE id = ?
  `;
  db.query(sql, [nombre, descripcion, id], (err, result) => {
    if(err){
      return res.status(500).json(err);
    }
    res.json({
      mensaje: 'Categoría actualizada'
    });
  });
});

// DELETE
router.delete('/:id', (req, res) => {
  const id = req.params.id;
  const sql = `
    DELETE FROM categorias
    WHERE id = ?
  `;
  db.query(sql, [id], (err, result) => {
    if(err){
      return res.status(500).json(err);
    }
    res.json({
      mensaje: 'Categoría eliminada'
    });
  });
});

```

```
module.exports = router;
```

Db.js

```
const mysql = require('mysql2');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'practica2'
});
connection.connect((err) => {
  if(err){
    console.log('Error de conexión');
  }else{
    console.log('Conectado a MySQL');
  }
});
module.exports = connection;
```

Server.js

```
const express = require('express');
const app = express();
app.use(express.json());
const categoriasRoutes = require('./routes/categorias');
app.use('/categorias', categoriasRoutes);
app.listen(3000, () => {
  console.log('Servidor corriendo en puerto 3000');
});
);
```

1. Crea un endpoint POST /categorias que permita registrar una nueva categoría enviando nombre y descripción en el body de la petición.

POST

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "nombre": "Ropa",
3   "descripcion": "Prendas de vestir"
4 }
```

Status: 200 OK Size: 152 Bytes Time: 133 ms

```
1 {
2   "mensaje": "Categoría registrada",
3   "result": {
4     "fieldCount": 0,
5     "affectedRows": 1,
6     "insertId": 3,
7     "info": "",
8     "serverStatus": 2,
9     "warningStatus": 0,
10    "changedRows": 0
11  }
12 }
```

2. Crea un endpoint GET /categorias que devuelva todas las categorías registradas en la base de datos.

```

1  [
2  {
3    "id": 1,
4    "nombre": "Electrónica",
5    "descripcion": "Dispositivos electrónicos y gadgets",
6    "createdAt": "2026-05-12T13:43:19.000Z",
7    "updatedAt": "2026-05-12T13:43:19.000Z"
8  },
9  {
10   "id": 2,
11   "nombre": "Oficina",
12   "descripcion": "Material y accesorios de oficina",
13   "createdAt": "2026-05-12T13:43:19.000Z",
14   "updatedAt": "2026-05-12T13:43:19.000Z"
15 },
16 {
17   "id": 3,
18   "nombre": "Ropa",
19   "descripcion": "Prendas de vestir",
20   "createdAt": "2026-05-12T14:10:15.000Z",
21   "updatedAt": "2026-05-12T14:10:15.000Z"
22 }
23 ]

```

3. Crea un endpoint GET /categorias/:id que devuelva la categoría con el ID especificado y además, incluya todos los productos que pertenecen a esa categoría.

```
GET http://localhost:3000/categorias/1

Query Headers Auth Body Tests Pre Run

Status: 200 OK Size: 165 Bytes Time: 7 ms

1  [
2    {
3      "id": 1,
4      "nombre": "Electrónica",
5      "descripcion": "Dispositivos electrónicos y gadgets",
6      "createdAt": "2026-05-12T13:43:19.000Z",
7      "updatedAt": "2026-05-12T13:43:19.000Z"
8    }
9  ]
```

4. Crea un endpoint PATCH /categorias/:id que permita actualizar todos los datos de la categoría con el ID especificado.

The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** http://localhost:3000/categorias/1
- Query:** (empty)
- Headers:** 2 (empty)
- Auth:** (empty)
- Body:** 1 (selected)


```
1 {
2   "nombre": "Tecnología",
3   "descripcion": "Equipos tecnológicos"
4 }
```
- Tests:** (empty)
- Pre Run:** (empty)
- JSON:** (selected)


```
1 {
2   "mensaje": "Categoría actualizada"
3 }
```
- XML:** (empty)
- Text:** (empty)
- Form:** (empty)
- Form-encode:** (empty)
- GraphQL:** (empty)
- Binary:** (empty)

Status: 200 OK **Size:** 36 Bytes **Time:** 72 ms

5. Crea un endpoint DELETE /categorias/:id que elimine la categoría indicada y, al mismo tiempo, elimine automáticamente todos los productos que pertenecen a esa categoría.

